四车协同 · 卷尺桥 · 架构方案

v0.2 · 4 台麦轮车 · 一辆车一个树莓派 · 放卷式卷尺伸出 + 电磁铁吸合 · 平行双跨

0 需求还原

0.1 初始排列与运动

```
初始（俯视）：4 台车一行并排，朝同一方向
  [1] [2] [3] [4]
   ↑   ↑   ↑   ↑

前两台往前开一段（拉开纵向距离），后两台原地不动：
  [1] [2]

  [3] [4]

四台各自原地转向对面：
  [1]→   ←[2]

  [3]→   ←[4]
```

0.2 最终形态：两段平行的"双跨"卷尺桥

```
俯视：

  [车1]═══卷尺═══[车2]      ← 前跨
                 ↑ 中间磁铁吸合
  [车3]═══卷尺═══[车4]      ← 后跨
```

0.3 已确认的约束

项	结论
底盘	麦轮，电机 **串口**驱动，需麦轮解算
定位	轮式编码器里程计 + IMU
俯仰舵机	**总线舵机**，只需开机设一个固定水平角 → **不是动态控制**
卷尺机构	**放卷式**：卷尺末端粘在圆盘上手动卷起，减速 PWM 电机驱动放卷
伸出长度	开环转圈数（**但需要"最后逼近"环节**，见 §4）
电磁铁	位于卷尺末端，两车面对面吸合
承重	**几百克以下** → 卷尺自重能撑，不需要额外张紧动作
主控	一辆车一个树莓派

运行	**直接上真车**（仿真跑不动，不做 Gazebo）

1 核心设计决策：事件驱动，不用延时

1.1 为什么 sleep 不成立

```
# ❌ 死程序
forward(speed); time.sleep(5); turn_left(90); time.sleep(2); extend_tape()
```

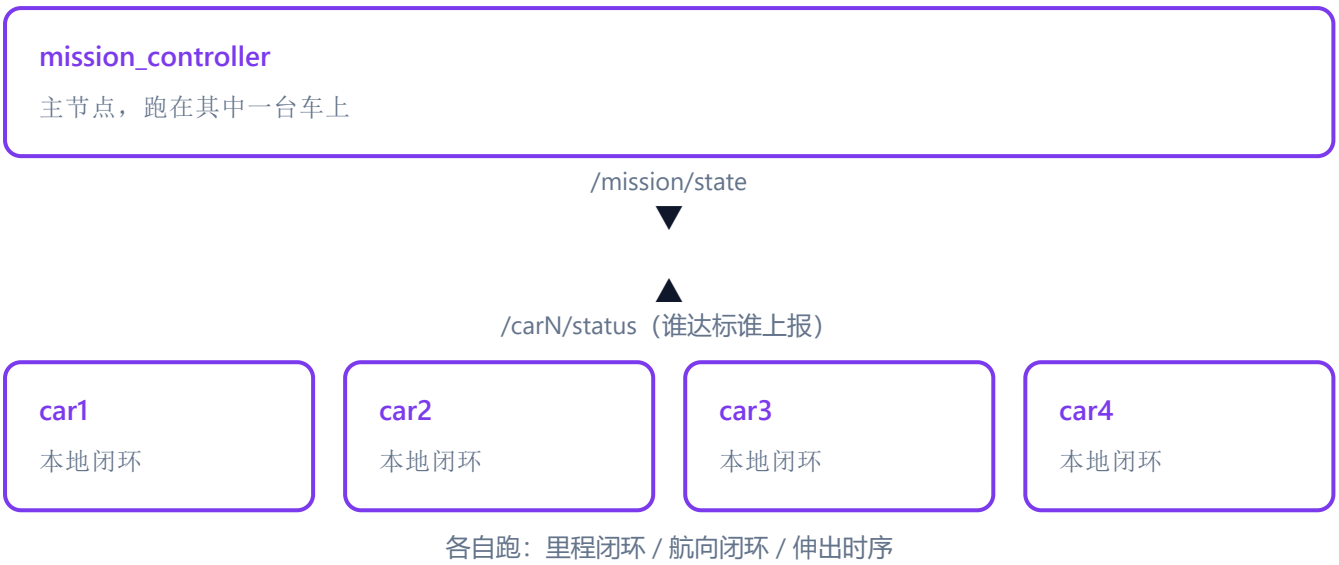
问题**不在 ROS**，在于**用时间当判据本身就不成立**：

干扰因素	后果
电池电量	满电 5 秒走 3.0m，电量掉一半可能只走 2.2m
地面摩擦 / 打滑	4 台车走同样时间，走的距离不同
4 台车电量不同	**4 台车进度不同步** ← 对"面对面吸合"是致命的
各车开机时间不同	多台树莓派的 `sleep` 出来的"同一时刻"物理上对不齐

1.2 三级方案对比

级别	做法	优点	缺点	建议
L0	每车烧死延时	最简单	上面那一堆问题	不用
L1	**条件驱动**：每个 sleep 换成"达到条件才走下一步"	改动最小，解决 90% 问题	仍是单机视角	必做
L2	**集中编排**：主机收集 4 车状态，集齐才发下一步	解决 4 车同步，不用时钟同步	有单点	必做
L3	ROS 2 **Action** 替代手写状态机	反馈/取消/超时现成	学习成本	留接口后加

1.3 推荐架构：集中编排 + 本地闭环



层	负责	判据
本地闭环（每台车）	走够距离、转到位、伸出、吸合	编码器里程、IMU 航向
集中编排（主节点）	什么时候允许进入下一阶段	集齐 4 台车的状态上报

> **关键收益：不需要时钟同步。**判断的是"状态"不是"时刻"，各车时钟差几秒完全无所谓。

2 系统架构

2.1 命名空间隔离

```
/car1/base_controller      底盘：串口驱动 + 麦轮解算 + 里程计
/car1/arm_controller       上层：舵机 + 卷尺电机 + 电磁铁
/car1/odom                 里程计 (nav_msgs/Odometry)
/car1/imu                  姿态   (sensor_msgs/Imu)
/car1/status               状态上报 (std_msgs/String)

/car2/...  /car3/...  /car4/...      同一套

/mission/state             编排层统一广播（所有车都订阅）
```

4 台车烧**同一个镜像**，靠环境变量决定自己是几号车：

```
export CAR_ID=2
ros2 launch four_car car.launch.py
```

2.2 跨机通信要点

- **必须设同一个** `ROS_DOMAIN_ID`（默认 0 够用）
- 建议固定用 **5GHz WiFi 或网线** —— 2.4G 在 4 台车同时跑时容易丢包
- 建议给每台车做 **DHCP 保留**（固定 IP），避免重启换 IP
- 选一个**低带宽的 RMW**，比如 `rmw_cyclonedds_cpp`

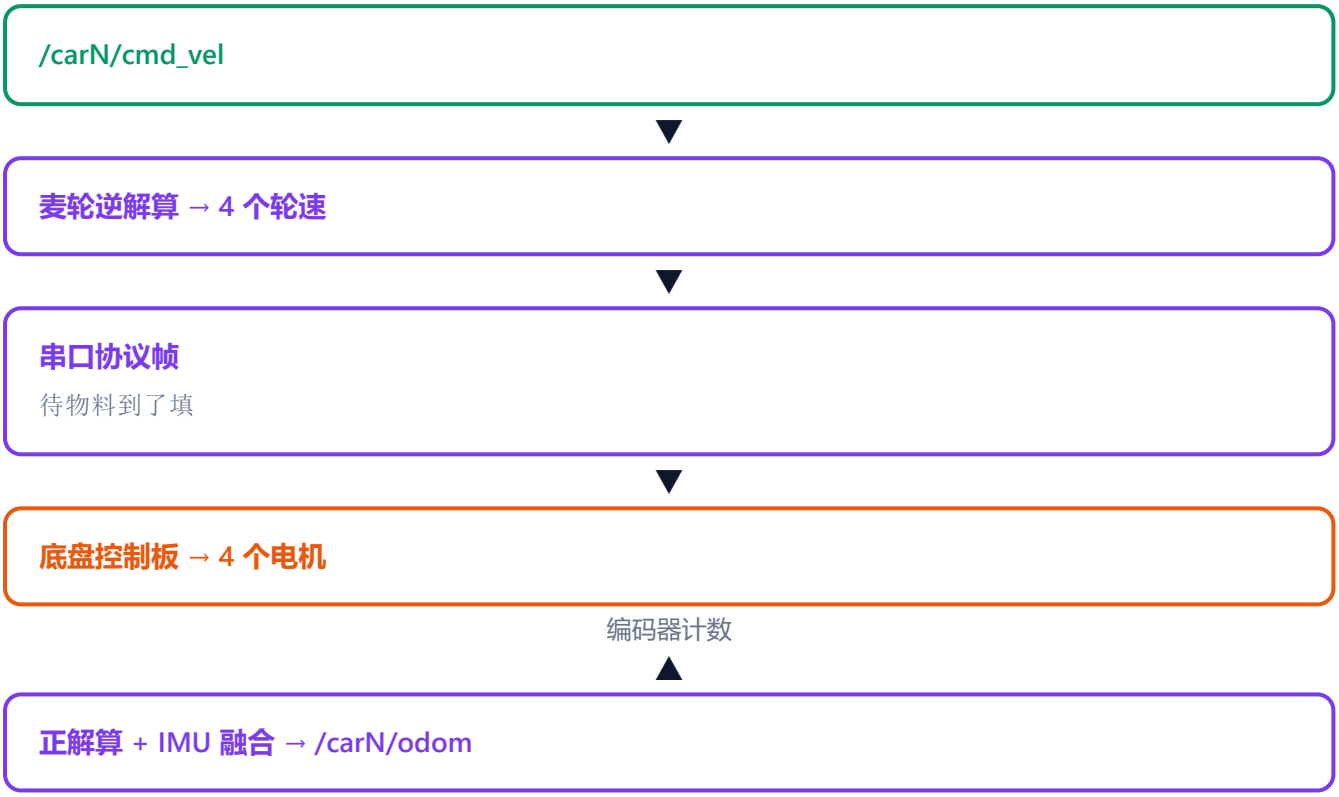
2.3 接口清单（草案）

话题	类型	方向	含义
`/mission/state`	`std_msgs/String`	编排 → 全部车	`IDLE` / `GO` / `EXTEND` / `DOCK` / `ABORT`
`/carN/status`	`std_msgs/String`	车 → 编排	`READY` / `MOVING` / `ALIGNED` / `EXTENDING` / `DOCKED` / `ERROR`
`/carN/cmd_vel`	`geometry_msgs/Twist`	车内部	速度指令 (vx, vy, wz)
`/carN/odom`	`nav_msgs/Odometry`	车内部	里程计（编码器 + IMU 融合）
`/carN/imu`	`sensor_msgs/Imu`	车内部	原始姿态

> 升级到 Action（L3）时，把 `/mission/state` 换成 `/carN/navigate_to` 的 goal 即可， 车内部接口**完全不用改**。

3 分层设计

3.1 底盘层 base_controller



> **为什么单独成层**：协议帧还没到，但 **解算和里程计逻辑与协议无关**，可以先写完用假数据单测。

3.2 上层 arm_controller

执行器	驱动方式	说明
俯仰舵机	总线舵机，`55 55` 串口协议	**只需开机设一个固定角度**，之后不用管 —— 直接复用 `servo_driver.py`
卷尺电机	GPIO PWM，减速电机	放卷式，开环 + 低速逼近（见 §4）
电磁铁	GPIO 开关量（待定）	最简单
霍尔传感器	GPIO 输入（推荐加）	用于"最后逼近"的到位检测（见 §4）

> **复用 servo_driver.py 要避开的坑**（今天已经踩过）：
>
> 1. `/dev/ttyAMA0` 被 `serial-getty` 占用 → `systemctl disable serial-getty@ttyXXX`
> 2. 用户不在 `dialout` 组 → `usermod -aG dialout`，且要退出重登
> 3. 读串口**不能傻等 timeout**，要"解析出帧立刻返回"，否则定时器回调会堆积
> 4. 一台车同时接**两个串口设备**（底盘板 + 舵机）→ 必须显式指定设备节点，**不要靠自动探测**

4 卷尺伸出机构（重点）

4.1 机构说明

卷尺末端粘在圆盘上、手动卷起，圆盘由**减速 PWM 电机**驱动**放卷**。

4.2 ⚠️ 放卷式机构的开环误差

每圈放出的长度不是常数 —— 随卷径减小而减小：

```
第 1 圈放出 = 2π × R     （R = 满卷半径，大）
   ...
第 N 圈放出 = 2π × r     （r = 卷芯半径，小）
```

举例：满卷 R=30mm、卷芯 r=10mm、转 10 圈

算法	结果	误差
按**满卷半径**算	2π×30×10 = **1885 mm**	+50%
按**卷芯半径**算	2π×10×10 = **628 mm**	−50%
按**中点半径 20mm**	2π×20×10 = **1257 mm**	≈ 真值

> **用"中点半径"估算几乎就是对的**（误差是二阶小量）。 但更靠谱的做法是**实测标定一张"圈数 → 伸出长度"对照表**，同一个机构标一次就够。

4.3 标定方法

```
① 理论估算：L(N) ≈ 2π × R_mid × N      （R_mid = (满卷半径 + 卷芯半径)/2）
② 实测：分别转 2/4/6/8/10 圈，用尺子量实际伸出长度
③ 拟合成查找表或线性函数，写进 params.yaml
④ 换卷尺/换机构后重标一次
```

4.4 到位判断方案（推荐）

> **为什么纯开环不够**：单边开环误差 1~3cm，两边加起来 **3~6cm 错位**， 而电磁铁吸力作用范围通常只有**几毫米到 1cm** → 大概率吸不上。

推荐：低速逼近 + 磁感应检测

① 两侧快速伸出到理论长度的 90~95%

▼

② 两侧同时低速继续伸

▼

③ 霍尔传感器检测到对面的磁铁（距离 < 1cm）→ 立刻停

▼

④ 两侧都停了 → 电磁铁通电 → 吸合

项	说明
成本	两个霍尔传感器（几毛钱）+ 卷尺末端一小块磁铁
优点	不硬撞、容差大（能吃掉 10cm 级开环误差）、失败可检测
配合	卷尺末端伸出口做**喇叭口/斜面导向**，允许几厘米横向错位被机械导入，最后由磁力自对准

不推荐的方案

方案	为什么不行
纯开环转够圈数就吸	3~6cm 错位大概率吸不上，**且失败不报错**（静默失败最难查）
末端限位开关硬碰停	两根卷尺正碰，机构要能承受撞击；且"撞上才触发"，已经撞了

4.5 承重分析

承重目标 **几百克以下**，卷尺自重能撑，**不需要额外的张紧动作。**

> 但仍需注意：
>
> - 卷尺中间的**挠度**（下垂）在小负载下可接受，但要在联调时实测确认
> - 两段接头处靠**磁铁吸力**传力，是整条桥最薄弱的地方 —— 建议实测"吸合后能承受多大侧向力"

5 任务时序状态机

5.1 每台车本地状态机

IDLE

收到 `/mission/state: GO`

▼

MOVING

前车(1,2)：直行到里程达标 | 后车(3,4)：直接进入转向

▼

转向

航向闭环，误差 < 2° 停

▼

ALIGNED

上报 → 等编排层

收到 /mission/state: EXTEND（编排层集齐 4 车 ALIGNED 才发）

▼

伸出

快速伸到 90% → 低速逼近 → 霍尔触发停 → 电磁铁通电 → 吸合

▼

DOCKED

上报 → 编排层集齐

5.2 编排层 mission_controller

IDLE → 广播 GO

等待 `/carN/status == ALIGNED` × 4

▼

EXTEND 广播

等待 `/carN/status == DOCKED` × 4（或按对 ×2）

▼

DONE

> **每一跳都是"条件"，没有一处是"时间"。**这是跟死程序的根本区别。

6 麦轮运动学

6.1 符号约定

```
轮序（俯视，车头朝上）          参数
      FL ●———● FR                vx  : 前进为正 (m/s)
         |    |                  vy  : 左移为正 (m/s)   ← 符号约定见 6.3
         |    |                  wz  : 逆时针为正 (rad/s)
      RL ●———● RR                r   : 轮半径 (m)
                                 L   : 半长 + 半宽 = Lx + Ly (m)
```

6.2 逆解算（速度 → 轮速）

```
w_FL = ( vx − vy − wz·L ) / r
w_FR = ( vx + vy + wz·L ) / r
w_RL = ( vx + vy − wz·L ) / r
w_RR = ( vx − vy + wz·L ) / r
```

6.3 正解算（轮速 → 速度）

```
vx = ( r / 4 )     · (  w_FL + w_FR + w_RL + w_RR )
vy = ( r / 4 )     · ( −w_FL + w_FR + w_RL − w_RR )
wz = ( r / (4L) )  · ( −w_FL + w_FR − w_RL + w_RR )
```

> **符号约定跟辊子朝向有关。**上面是"45° 辊子、X 型布局"的标准式， 不同厂家的辊子方向可能相反，`vy` 和 `wz` **的符号要对实物标定。**

6.4 标定方法（3 分钟）

测试	发什么	期望	不对就改
前进	`vx=0.2, vy=0, wz=0`	车头方向直着走	整体符号
横移	`vx=0, vy=0.2, wz=0`	往**左**平移	翻 `vy` 符号
自转	`vx=0, vy=0, wz=0.5`	**逆时针**转	翻 `wz` 符号

6.5 本项目实际只用到的动作

> 好消息：**这个任务不需要全向运动。**实际只用到三种：
>
> | 动作 | 用什么 |
> |---|---|
> | 直行一段 | `vx = +`，`vy = wz = 0` |
> | 原地转 90° | `vx = vy = 0`，`wz = ±` |
> | （可选）微调对准 | 小幅度 `vy` 横移 |
>
> 麦轮解算仍按通用的写（方便扩展），但**调试时不必四个方向都标定，先把这三个跑通。**

7 目录结构

```
four_car_ws/
├── src/
│   ├── car_base/                    底盘层
│   │   ├── car_base/base_driver.py        串口协议（待填）
│   │   ├── car_base/mecanum.py            麦轮正逆解算   ← 现在就能写
│   │   ├── car_base/odometry.py           里程计 + IMU 融合
│   │   └── car_base/base_node.py          ROS 节点
│   ├── car_arm/                     上层
│   │   ├── car_arm/servo_driver.py        ← 从舵机项目搬过来复用
│   │   ├── car_arm/tape_driver.py         放卷电机 + 标定表 + 逼近逻辑
│   │   ├── car_arm/magnet.py              电磁铁 + 霍尔检测
│   │   └── car_arm/arm_node.py
│   ├── car_agent/                   单车的本地状态机
│   ├── mission_controller/          集中编排
│   └── four_car_bringup/
│       ├── launch/car.launch.py           单车启动（靠 CAR_ID 区分）
│       ├── launch/mission.launch.py       编排层启动
│       └── config/params.yaml             所有可调参数集中在这
└── docs/
    └── 架构方案.md
```

> **参数全部集中在** `config/params.yaml` —— 改角度、改距离、改速度只动这一个文件。

8 待你确认 / 提供的清单

8.1 必须的（没有就写不了驱动）

- ☐ **底盘电机串口协议**：波特率、帧格式、速度指令怎么表示、有没有编码器回读
- ☐ **底盘板连接方式**：USB 转串口（`/dev/ttyUSB0`）还是 GPIO 串口（`/dev/ttyAMA0`）
- ☐ **总线舵机的串口**：和底盘**必须分开用两个设备节点**
- ☐ **麦轮参数**：轮半径 `r`、半长 `Lx`、半宽 `Ly`
- ☐ **PWM 卷尺电机的驱动方式**：GPIO 直驱还是电调；频率、占空比范围、正反转怎么控制
- ☐ **电磁铁**：电压/电流、继电器还是 MOSFET、GPIO 引脚
- ☐ **霍尔传感器型号和接线**（如果采纳 §4.4 方案）

8.2 任务参数

- ☐ 前两台车要直行多少距离？（米）
- ☐ 转向是**原地转**还是边走边转？
- ☐ 卷尺**总长**多少？（决定最大跨度）
- ☐ 两车最终间距多少？（= 卷尺伸出长度）
- ☐ 卷芯半径 / 满卷半径（用于标定表初值）

8.3 我还没问的

- ☐ **4 台树莓派有了吗？**型号是否一致？（手上那台是 Pi 5，不同型号要统一 ROS 2 版本）
- ☐ **IMU 型号**？接在底盘板上，还是单独一个走 I2C/串口？
- ☐ **编码器**：底盘板能读回来吗？还是轮子上单独装？
- ☐ **急停**需求？（4 台车在桌上跑，出问题要能一键停）

9 风险与对策

风险	影响	对策
放卷机构开环误差	两侧累计 3~6cm 错位 → 吸不上	§4.4 低速逼近 + 霍尔检测
电磁铁吸合是薄弱环节	承重时接头先失效	末端做喇叭口导向；实测接头能承受的侧向力
IMU 的 yaw **会漂**	转 90° 越转越偏	只用短时间闭环（转到目标附近就停），别长时间积分
WiFi 多机丢包	状态上报丢 → 编排层永远等不齐	状态上报**周期性重发**；编排层加超时
4 车电量不一致	同样的闭环精度，快的先到	已由"条件驱动"化解；再加"到位后原地等待"
单点故障（主机挂）	全任务停	主机选电量最足的；或降级为各车本地跑 L1 时序
地面摩擦不同	横移/直行时车会偏航	四轮闭环 + IMU 航向修正
两车四轮对不齐	卷尺伸出方向偏	转向用航向闭环，误差 < 2° 才停

10 下一步

1. **本文档确认后**，先写**不依赖硬件**的部分：
   - `mecanum.py` 麦轮正逆解算（可用假数据单测）
   - `odometry.py` 里程计 + IMU 融合
   - `tape_driver.py` 的标定表 + 低速逼近逻辑
   - 状态机与编排层骨架
   - 所有驱动的**接口定义**（空实现）
2. 物料到了，按 §8 填实现，接上就能跑
3. 树莓派那边先把**串口环境**配好（getty / dialout 坑），4 台都要配